

Projeto de Desenvolvimento de Software

Briefing de Projeto (L-EI)

Licenciatura em Engenharia Informática
2026-2027 / 5º Semestre

PROJETO MULTIDISCIPLINAR:

Projeto de Desenvolvimento de Software (Rui Pascoal/ Miguel Boavida,
rui.pascoal@universidadeeuropeia.pt; miguel.boavida@universidadeeuropeia.pt)

Engenharia de Software (Rui Ramos, rui.ramos@universidadeeuropeia.pt)

Segurança Informática (Sérgio Nunes/ Pedro Brandão,
sergio.nunes@universidadeeuropeia.pt; pedro.r.brandao@universidadeeuropeia.pt)

Sistemas Distribuídos (Pedro Rosa, pedro.rosa@universidadeeuropeia.pt)

Inteligência Artificial (Samuel Gomes, samuel.s.gomes@universidadeeuropeia.pt)



**Faculdade de Design,
Tecnologia e Comunicação**
Universidade Europeia

01_ DESCRIÇÃO DO PROJECTO

No contexto da L-El pretende-se que os alunos desenvolvam um projeto multidisciplinar, integrando os conhecimentos e aprendizagens adquiridos em várias unidades curriculares do 5º semestre do curso.

O projeto consiste na proposta e desenvolvimento de uma solução baseada numa arquitetura distribuída que resolva uma necessidade real para a qual a distribuição é requerida. A proposta de arquitetura tem de ser validada nas UC's de Projeto e Sistemas Distribuídos. A arquitetura pode integrar elementos de IoT e ter nós baseados em microcontrolador.

A solução deve também incluir outros componentes, nomeadamente componentes de Inteligência Artificial e Segurança Informática.

A demonstração da solução deve evidenciar a natureza distribuída do sistema, a sua segurança e tolerância a falhas.

02_ PROCESSO DE DESENVOLVIMENTO

Este projeto baseia-se na análise, desenho e implementação de um Projeto de Engenharia Informática. Os estudantes são assim guiados através de todo o processo de desenvolvimento, recorrendo a uma metodologia ágil, que culmina com uma apresentação dos resultados.

As diversas fases incluem, obrigatoriamente:

- Análise e conceptualização, nas diversas áreas do projeto.
- Planeamento.
- Prototipagem.
- Implementação.
- Teste.
- Apresentação.

03_ OBJECTIVOS

No final da unidade curricular, o aluno deverá ser capaz de:

- Capaz de compreender as principais problemáticas associadas à distribuição, segurança e tolerância a falhas em sistemas informáticos.
- Capaz de desenhar uma arquitetura distribuída de suporte a uma solução informática.
- Integrar mecanismos de Inteligência Artificial na solução.
- Compreender as diferentes tecnologias e etapas envolvidas no desenvolvimento do projeto.
- Trabalhar no desenvolvimento de um projeto através de um sistema/plataforma colaborativa de controlo de versões.
- Utilizar uma metodologia de desenvolvimento ágil, e produzir artefactos auditáveis.

04_ REQUISITOS

O projeto deve ter em conta os seguintes requisitos obrigatórios:

- Os grupos têm uma dimensão máxima de 4 (quatro) elementos. Exceções a esta regra podem existir, desde que aprovadas pelo professor de Projeto.
- Em avaliação final o projeto é individual e será realizado dentro do período reservado para o exame final de 48 horas.
- A escolha do tema do projeto é da responsabilidade dos grupos de trabalho, sujeita a validação pelos docentes das unidades curriculares envolvidas.
- Não existem limitações quanto a linguagens e tecnologias a utilizar, recomenda-se a utilização de ferramentas e linguagens dadas até ao momento e em particular as tecnologias utilizadas durante o presente semestre. Cada grupo deve realizar uma proposta de *stack* tecnológica e tem de ser validada pelos docentes envolvidos.
- É obrigatório utilizar a seguinte plataforma (a avaliação tem em conta a interação com esta plataforma, sendo que é usada para monitorização contínua do projeto):
 - GitHub para o versionamento do código e o acompanhamento do desenvolvimento.
- Na documentação deve ser incluído toda a informação de suporte à concessão, desenho e implementação da arquitetura da solução (distribuída).

05_ CONTRIBUIÇÕES DAS UCs

ECTS	HORAS /SEMANA	CONTRIBUIÇÕES DE UCS	DOCENTE	% DO PROJ. FINAL
6	3h (14w)	Engenharia de Software Aplicação de princípios e metodologias de Engenharia de Software na análise do problema, identificação e especificação de requisitos bem como na modelação da solução. Os grupos deverão traduzir as necessidades identificadas numa proposta de software coerente, considerando os principais utilizadores, funcionalidades e requisitos do sistema e utilizando técnicas de modelação adequadas para apoiar o seu desenvolvimento.	Rui Ramos	30% Av. Contínua
6	3h (14w)	Segurança Informática Componentes de segurança.	Sérgio Nunes/ Pedro Brandão	10% Av. Contínua
6	3h (14w)	Sistemas Distribuídos Descrição genérica da solução a implementar enquadrada nas áreas da Unidade Curricular, identificando as necessidades de prestação do serviço de forma distribuída na ótica de concorrência e tolerância a faltas; Descrição dos requisitos Técnicos para desenvolvimento do projeto, incluindo: Arquitetura	Pedro Rosa	50% Av. Contínua

		da Solução; Tecnologias a utilizar para distribuição dos componentes envolvidos (podem ser alteradas/otimizadas durante o andamento do projeto, nomeadamente até à segunda entrega); Suporte existente à tolerância a faltas, enquadrado na Arquitetura da Solução. Implementação da Arquitetura distribuída e tolerante a faltas, garantindo um nível de disponibilidade e integridade adequado ao projeto de desenvolvimento de software. Demonstração prática dos mecanismos de tolerância a faltas, garantindo que, com a introdução de problemas propositados de disponibilidade de componentes, a solução mantenha a disponibilidade e integridade adequada.		
6	3h (14w)	Inteligência Artificial A aplicação deve integrar pelo menos uma componente de IA, escolhida entre os tópicos lecionados na UC, incluindo: - Algoritmos de Procura Não Informada e Informada – por exemplo, pesquisa em grafos (BFS, DFS, A*) para calcular caminhos ótimos ou encontrar soluções em problemas de navegação, planeamento ou logística. Em cibersegurança, podem-se representar computadores como nós e as ligações entre eles como arestas, e um algoritmo de procura pode ser utilizado para prever e bloquear possíveis caminhos que um atacante poderia seguir para chegar a um computador crítico. - Problemas de Satisfação de Restrições (CSP) – para resolver tarefas como agendamento de recursos, alocação de salas, gestão de stocks ou resolução de puzzles com restrições. Pode ser usado para definir e otimizar a alocação de recursos dadas regras e restrições de segurança (por exemplo, restrições de acesso). - Procura com Adversários (ex.: Minimax com poda Alpha-Beta) – aplicável a módulos de jogo, simulações competitivas ou decisões estratégicas entre dois lados. Em cibersegurança, pode simular situações em que existem dois lados: um atacante e um sistema de defesa que operam num mesmo meio computacional, e ser usado para escolher a melhor estratégia de defesa dada uma ação do utilizador. - Algoritmos de Aprendizagem Automática: Machine Learning (ML) – processos automáticos que aprendem a tomar decisões certas, sempre que existam dados ou simulações para treino (ex.: otimizar estratégias num ambiente dinâmico ou simulado). Neste caso, um processo destes pode ser treinado para reconhecer acessos indesejados ou anomalias.	Samuel Gomes	60% Av. Continua

06_ ENQUADRAMENTO E TRABALHOS PROPOSTOS

Todos os trabalhos propostos neste projeto têm como objetivo a concepção e implementação de um sistema distribuído tolerante a faltas, no qual os diferentes componentes do sistema são executados de forma distribuída e cooperam através de uma rede de comunicação.

Embora cada trabalho apresente um problema e uma aplicação diferentes, existe um conjunto de requisitos e objetivos comuns. Em particular, pretende-se que os alunos sejam capazes de identificar os pontos únicos de falha de uma solução e conceber mecanismos de redundância e recuperação que permitam manter o serviço disponível perante a falha de um ou mais componentes.

A solução deverá, sempre que tecnicamente aplicável, considerar a redundância das diferentes camadas que constituem o sistema, nomeadamente:

- **Camada aplicacional / cliente:** deverá ser possível disponibilizar mais do que uma instância da aplicação cliente ou mecanismos alternativos de acesso ao serviço, quando aplicável;
- **Camada de serviços / backend:** os serviços responsáveis pelo processamento dos pedidos deverão ser executados em múltiplas instâncias e/ou máquinas, evitando que a falha de um servidor provoque a indisponibilidade do sistema;
- **Base de dados:** sempre que exista uma base de dados central, esta não deverá constituir um ponto único de falha. Deverão ser estudadas e, sempre que possível, implementadas soluções de replicação e recuperação;

A redundância não deverá ser entendida apenas como a existência de várias máquinas. Os alunos deverão justificar onde é necessária a replicação, que tipo de falhas se pretende suportar e de que forma o sistema deteta e reage a essas falhas.

06_01_Arquitetura Distribuída

A arquitetura deverá ser apresentada através de um ou mais diagramas, que podem identificar os seguintes fatores (não obrigatórios na totalidade):

- clientes;
- servidores e serviços;
- bases de dados e respetivas réplicas;
- componentes responsáveis pela deteção de falhas;
- mecanismos de replicação e sincronização;
- mecanismos de recuperação.

Deverão ser justificadas as principais decisões arquiteturais, nomeadamente as relacionadas com replicação, consistência, disponibilidade, deteção de falhas e recuperação.

06_02_Tolerância a Faltas

Para cada projeto deverá ser definido um modelo de faltas, no primeiro relatório entregue, identificando pelo menos (onde aplicável):

- quais os componentes que podem falhar;
- que tipos de falha são considerados;
- quantas falhas simultâneas o sistema deverá conseguir suportar;
- quais as funcionalidades que deverão permanecer disponíveis após uma falha;
- como é detetada uma falha;
- como é realizada a recuperação ou substituição do componente;
- que informação pode ser perdida, caso alguma, durante uma falha;
- como é garantida a consistência dos dados entre as diferentes réplicas.

No final, deverá ser possível demonstrar experimentalmente a tolerância a faltas através de testes de falha, por exemplo desligando um container ou servidor, interrompendo um serviço, tornando uma réplica da base de dados indisponível ou interrompendo temporariamente uma comunicação.

O objetivo não é necessariamente garantir que o sistema continue a funcionar perante qualquer tipo ou número de falhas. Pretende-se que os alunos definam claramente as garantias oferecidas pelo sistema e demonstrem que estas são efetivamente cumpridas.

06_03_Demonstração de Tolerância a Faltas

A avaliação do projeto deverá incluir uma demonstração prática da tolerância a faltas da solução implementada.

Os alunos deverão identificar previamente um conjunto de cenários de falha relevantes para o sistema e demonstrar o seu comportamento perante essas falhas. Deverão ser considerados, sempre que aplicável, cenários como:

- falha de uma instância do serviço aplicacional;
- falha de um servidor de backend;
- falha de uma réplica da base de dados;
- interrupção da comunicação entre componentes;
- perda de um nó de armazenamento;
- recuperação de um componente após uma falha.

Para cada cenário deverá ser indicado:

1. componente que falha;
2. como a falha é detetada;
3. que mecanismos permitem manter o serviço disponível;
4. qual o impacto sobre os utilizadores;
5. se existe perda de dados;
6. como é realizada a recuperação;
7. como é restabelecida a redundância após a recuperação.

A existência de múltiplas réplicas, por si só, não será considerada suficiente para demonstrar tolerância a faltas. Deverá ser demonstrado experimentalmente que, perante a falha de um componente, os restantes componentes conseguem assumir as suas funções de acordo com as garantias definidas no projeto.

06_04__Trabalhos Propostos

Os projetos são exemplificativos. Os alunos podem escolher outros projetos, desde que aprovados pelos Docentes.

Trabalho 1 — Sistema Distribuído de Sensores para Monitorização de Trânsito

A condução autónoma e os sistemas avançados de assistência ao condutor dependem da disponibilidade de grandes quantidades de informação sobre o ambiente envolvente, incluindo informação sobre tráfego, condições meteorológicas, acidentes, estado das vias e outros eventos relevantes.

Pretende-se desenvolver um sistema distribuído de recolha, armazenamento e disponibilização de dados provenientes de sensores, simulando uma infraestrutura de monitorização de trânsito.

O sistema deverá receber informação proveniente de múltiplos sensores, armazená-la de forma distribuída e tolerante a faltas e disponibilizar posteriormente essa informação através de uma API ou webservice. Os clientes poderão representar veículos, aplicações de navegação ou outros sistemas consumidores de informação.

Os dados poderão ser obtidos através de dispositivos reais, por exemplo ESP32, ou gerados por aplicações que produzam dados em tempo real de acordo com regras previamente definidas. Deverá ser possível simular a existência de vários sensores distribuídos geograficamente.

A solução deverá considerar a redundância dos serviços de processamento e do armazenamento dos dados, bem como os mecanismos necessários para garantir a continuidade do serviço perante a falha de componentes.

Exemplos de dados: velocidade média, número de veículos, congestionamentos, temperatura, precipitação, visibilidade, acidentes ou estado do pavimento.

Extensões possíveis:

- processamento de eventos em tempo real;
- deteção automática de congestionamentos;
- localização geográfica dos sensores;
- dashboard de monitorização;
- utilização de MQTT;

Trabalho 2 — Sistema de Votação On-Line Distribuído

A disponibilização de sistemas de votação através da Internet coloca desafios significativos relacionados com segurança, disponibilidade, integridade, autenticidade e confiança.

Pretende-se desenvolver um sistema distribuído de votação on-line, permitindo que utilizadores autenticados participem numa ou mais eleições através de uma aplicação web ou, opcionalmente, de uma aplicação Android.

O sistema deverá possuir uma arquitetura distribuída, com múltiplas instâncias dos serviços responsáveis pela gestão das eleições e pelo processamento dos votos. Os dados deverão ser

armazenados de forma redundante, através de bases de dados replicadas, não devendo existir um único servidor cuja falha comprometa a disponibilidade do sistema.

Toda a comunicação deverá ser protegida através de mecanismos criptográficos adequados, nomeadamente HTTPS. Os dados armazenados deverão igualmente ser protegidos, devendo os alunos definir quais os dados que necessitam ser cifrados e como poderá ser gerida essa proteção (a possibilidade de implementação de cifra na BD deverá ser analisada durante as aulas).

O sistema deverá manter um registo de auditoria das operações relevantes, permitindo identificar as ações realizadas pelos diferentes componentes e utilizadores. Este mecanismo de logging deverá, tanto quanto possível, ser independente e tolerante à falha de componentes individuais – esta funcionalidade poderá ser analisada em conjunto com algum grupo que, eventualmente, efetue a gestão de logs distribuídos.

Um aspeto particularmente importante será a definição das propriedades de segurança e confiança que o sistema pretende garantir. Os alunos deverão explicitar, por exemplo, como são assegurados a autenticação dos eleitores, a integridade dos votos, a prevenção de votos duplicados e a separação entre a identidade do eleitor e o seu voto, quando aplicável.

O contexto concreto da votação — eleição académica, votação numa organização, eleição fictícia, etc. — ficará ao critério dos alunos e deverá ser definido na proposta de trabalho.

Trabalho 3 — Repositório de Ficheiros Distribuído On-Line

Serviços como Dropbox, Google Drive e OneDrive permitem aos utilizadores armazenar e aceder aos seus ficheiros a partir de diferentes dispositivos e localizações. A disponibilização destes serviços exige mecanismos de armazenamento distribuído, replicação e elevada disponibilidade.

Pretende-se desenvolver um repositório de ficheiros distribuído, acessível através de uma aplicação web e, opcionalmente, de uma aplicação móvel.

O sistema deverá permitir, pelo menos:

- autenticação dos utilizadores;
- carregamento e descarregamento de ficheiros;
- criação e organização de diretórios;
- consulta dos ficheiros disponíveis;
- eliminação e recuperação de ficheiros;
- armazenamento dos ficheiros de forma redundante.

A arquitetura deverá evitar pontos únicos de falha, devendo existir redundância tanto nos serviços responsáveis pelo acesso aos ficheiros como na infraestrutura de armazenamento.

Os alunos deverão definir a estratégia de replicação dos ficheiros, considerando questões como número de réplicas, localização das réplicas, sincronização e recuperação após a falha de um nó.

Sempre que possível, os ficheiros deverão ser armazenados de forma cifrada, devendo ser analisadas as implicações dessa opção ao nível do desempenho, gestão de chaves e recuperação.

Extensões possíveis:

- versionamento de ficheiros;
- partilha de ficheiros entre utilizadores;
- sincronização automática;
- deteção de ficheiros duplicados;
- armazenamento por *chunks*;
- replicação entre diferentes nós;
- recuperação automática de uma réplica perdida.

Trabalho 4 — Sistema Distribuído de Deteção e Auto-Reposição de Ficheiros / Anti-Ransomware

A alteração ou destruição não autorizada de ficheiros pode resultar de falhas humanas, erros de software ou ataques informáticos, incluindo ataques de ransomware.

Pretende-se desenvolver um sistema distribuído de monitorização e proteção de ficheiros, capaz de detetar alterações consideradas anómalas ou não autorizadas e, quando necessário, recuperar automaticamente uma versão anterior dos ficheiros.

Os alunos deverão começar por definir e justificar o mecanismo de deteção a utilizar. Poderão, por exemplo, ser considerados mecanismos baseados em *hashes*, monitorização de alterações ou combinação de vários indicadores.

O sistema deverá manter informação suficiente para permitir a recuperação dos ficheiros após uma alteração considerada indevida. Essa informação deverá ser armazenada de forma redundante, de modo a que a própria infraestrutura de recuperação não constitua um ponto único de falha.

Deverá ainda ser considerada a possibilidade de um atacante comprometer o sistema que está a ser protegido. Assim, sempre que possível, os mecanismos de deteção e as cópias de recuperação deverão estar isolados do sistema monitorizado.

O trabalho deverá incluir testes que simulem diferentes tipos de alterações, permitindo avaliar a capacidade do sistema para distinguir alterações legítimas de comportamentos potencialmente maliciosos.

Extensões possíveis:

- deteção de comportamento semelhante a ransomware;
- *file versioning*;
- cópias imutáveis;
- armazenamento *offline* ou isolado;
- recuperação automática;
- alertas para o administrador;
- análise distribuída de eventos.

Trabalho 5 — Sistema Distribuído de Autenticação de Dois Fatores (2FA)

A utilização exclusiva de nome de utilizador e palavra-passe apresenta riscos de segurança significativos. Os sistemas de autenticação de dois fatores acrescentam um segundo mecanismo de validação, como códigos temporários, aplicações móveis, notificações ou dispositivos físicos.

Pretende-se desenvolver um sistema distribuído de autenticação de dois fatores, preferencialmente acompanhado por uma aplicação Android que funcione como segundo fator de autenticação ou utilizando o Google Authenticator ou app semelhante.

O sistema deverá disponibilizar uma infraestrutura redundante para os serviços de autenticação, evitando que a falha de um único servidor impossibilite a autenticação dos utilizadores.

Os alunos deverão implementar diferentes mecanismos de segundo fator (como por exemplo, mas não só, utilização de TOTP - Time-based One-Time Password - através da geração de códigos temporários numa aplicação móvel).

A solução deverá considerar igualmente a redundância e disponibilidade dos dados necessários ao processo de autenticação, bem como os mecanismos de recuperação perante falhas.

Deverão ser analisados aspetos como:

- armazenamento seguro das credenciais;
- armazenamento seguro dos segredos utilizados pelo segundo fator;
- proteção das comunicações;
- sincronização temporal;
- prevenção de reutilização de códigos;
- recuperação de contas;
- comportamento do sistema perante a falha de um servidor de autenticação.

Trabalho 6 — Sistema Distribuído de Armazenamento e Gestão de Logs

O registo das operações realizadas por sistemas e aplicações é fundamental para a monitorização, auditoria, deteção de incidentes e implementação de políticas de segurança.

Pretende-se desenvolver um sistema distribuído de recolha, armazenamento e consulta de logs, capaz de receber informação proveniente de diferentes aplicações e dispositivos e armazená-la de forma distribuída e replicada.

O sistema deverá disponibilizar mecanismos para a receção de mensagens de log através de diferentes meios, que poderão incluir APIs REST, webservices, protocolos de mensagens ou outros mecanismos definidos pelos alunos.

Os dados recebidos deverão ser encaminhados para uma infraestrutura de armazenamento redundante. A arquitetura deverá evitar que a falha de um servidor de recolha ou armazenamento provoque a perda dos logs ou a indisponibilidade completa do serviço.

Para demonstrar o funcionamento da solução, os alunos deverão desenvolver pequenas aplicações de teste capazes de gerar e enviar diferentes tipos de eventos.

Deverão ser realizados testes de falha, verificando, por exemplo, o comportamento do sistema quando um servidor de recolha ou um nó de armazenamento deixa de estar disponível.

Extensões possíveis:

- compatibilidade syslog;
- pesquisa e filtragem de logs;
- dashboard;
- alertas;
- deteção de padrões;
- correlação de eventos;
- diferentes níveis de severidade;

Trabalho 7 — Sistema Distribuído de Monitorização de Infraestruturas

Desenvolvimento de uma plataforma semelhante a um pequeno sistema de *monitoring*, recolhendo CPU, memória, disco, rede, disponibilidade de serviços, etc., a partir de vários computadores.

O desafio distribuído seria particularmente interessante porque permite trabalhar agentes, recolha distribuída, armazenamento, replicação, deteção de falhas e alertas.

Trabalho 8 — Serviço de Mensagens Distribuído

Desenvolvimento de um sistema de mensagens instantâneas em que os utilizadores possam comunicar através de clientes web ou móveis.

A infraestrutura deverá suportar múltiplos servidores e garantir que as mensagens não são perdidas quando um dos servidores falha.

Permite trabalhar conceitos de filas, comunicação assíncrona, persistência, replicação, consistência e recuperação.

Trabalho 9 — Sistema Distribuído de Partilha de Sensores / IoT

Uma evolução do trabalho dos sensores de trânsito: vários dispositivos IoT publicam informação numa infraestrutura distribuída, que recebe, processa, armazena e disponibiliza os dados.

Pode ser particularmente interessante para trabalhar MQTT, *publish/subscribe*, brokers redundantes, processamento de eventos e armazenamento distribuído.

Trabalho 10 — Sistema Distribuído de Reservas

Um sistema de reservas de recursos — salas, lugares, equipamentos, estacionamento, bilhetes, etc. — disponibilizado através de uma aplicação web e/ou mobile.

Aqui o desafio principal seria a consistência distribuída: dois utilizadores não podem, por exemplo, reservar simultaneamente o mesmo recurso.

É um trabalho particularmente interessante porque permite explorar concorrência, transações, replicação, conflitos e consistência.

Trabalho 11 — Plataforma Distribuída para Mobile Ethnography e Diary Studies

Mobile Ethnography é uma abordagem de investigação em que os participantes utilizam o seu smartphone para registar e partilhar experiências no contexto em que estas acontecem. Diary Studies são estudos em que os participantes fazem esses registos ao longo de um determinado período, seguindo atividades ou perguntas definidas pelo investigador. Estas abordagens podem ser utilizadas, por exemplo, para estudar experiências de consumidores, utilização de produtos ou serviços, comportamentos no quotidiano ou experiências em determinados espaços.

O objetivo é desenvolver uma plataforma distribuída que permita a investigadores criar estudos, definir atividades e acompanhar participantes em tempo real. Através de uma aplicação móvel, os participantes deverão poder receber atividades e registar as suas experiências através de

texto, fotografia, vídeo ou áudio, podendo estes registos incluir informação como localização, data e hora.

Por exemplo, um estudo pode pedir aos participantes para fotografarem um produto que utilizam diariamente, registarem a experiência de utilizar um serviço, documentarem um percurso pela cidade ou responderem regularmente a perguntas sobre determinada atividade.

A plataforma deverá permitir gerir investigadores, estudos, atividades, participantes e registos, bem como consultar, organizar e exportar os dados recolhidos. A solução deverá ser pensada para suportar vários estudos e participantes em simultâneo.

Cada grupo deverá definir as funcionalidades, arquitetura, tecnologias e modelo de dados da solução, justificando as principais decisões tomadas.

07_ METODOLOGIA, ENTREGAS & AVALIAÇÃO

O projeto será desenvolvido ao longo do semestre com a colaboração das diferentes unidades curriculares envolvidas. Cada unidade curricular faz o acompanhamento dos trabalhos para cada entrega, e a validação dos respetivos entregáveis.

Para cumprir com o método de avaliação contínua, os estudantes têm de efetuar 3 entregas. Existe um mínimo de 70% de assiduidade (inferior apenas para casos especiais previstos no regulamento) para que os alunos sejam elegíveis para avaliação contínua.

A avaliação de cada estudante é efetuada de acordo com o trabalho realizado, conforme descrito abaixo. Existe uma penalização de até 4 pontos se as instruções de entrega não forem seguidas com precisão.

1ª Entrega:

€ Data de Submissão: 11-10-2026 às 23h59

€ **Data da Apresentação: 09-10-2026**

€ Entregável:

- Memória e arquivo documental do projeto (ver secção 08).
Primeira versão do relatório final, disponibilizada no Canvas (em pdf).
Modelo de Faltas
- Suporte visual da apresentação do projeto (slides em pdf).

- Avaliação (20%)

2ª Entrega:

● Data de Submissão: 15-11-2026 às 23h59

● **Data da Apresentação: 15-11-2026**

● Entregáveis:

- Memória e arquivo documental do projeto atualizada até à data (ver secção 08).
- Protótipo do projeto (no Github).
Segunda versão do relatório final, disponibilizada no Canvas (em pdf).
- Suporte visual da apresentação do projeto (slides em pdf).

€ Avaliação (30%)

3ª Entrega:

● Data de Submissão: 20-12-2026 às 23h59

● **Data da Apresentação: 18-12-2026**

● Entregáveis:

- Memória e arquivo documental do projeto atualizada até à data (ver secção 08).
- Protótipo do projeto (no Github).
versão final do relatório final, disponibilizada no Canvas (em pdf).
- Suporte visual da apresentação do projeto (slides em pdf).

- Avaliação (50%):

Caso os prazos não sejam cumpridos:

A avaliação de cada estudante é efetuada de acordo com o trabalho realizado, conforme descrito abaixo. Existe uma penalização de até 4 pontos se as instruções de entrega não forem seguidas com precisão.

Avaliação Final / Exame:

- Descrição técnica e conceptual do projeto (60%)
- Implementação e documentação do projeto (40%)

08_ MEMÓRIA E ARQUIVO DOCUMENTAL DO PROJETO

Esta documentação tem como principal objetivo garantir a longevidade e o arquivo histórico dos projetos desenvolvidos pelos alunos no meio académico. Estes documentos posteriormente serão partilhados com a coordenação do modelo académico e servirão como base para exposições, livros, entre outros outputs.

Vale ressaltar que esta documentação e sua organização e formato devem seguir estritamente as diretrizes apontadas abaixo, não havendo abertura para criatividade ou alterações no formato solicitado.

CONJUNTO MÍNIMO E OBRIGATÓRIO DE DOCUMENTOS

Cada projeto deverá disponibilizar, no mínimo:

1. Identificação dos Estudantes: Para cada estudante:
 - a. Nome completo
 - b. Número de estudante
 - c. Curso
 - d. Ano curricular
 - e. Contacto institucional (quando aplicável)
2. Memória Descritiva: Documento produzido pelos estudantes contendo:
 - a. Identificação
 - i. Nome do projeto
 - ii. Ano letivo
 - iii. Semestre
 - iv. Unidade(s) curricular(es)
 - v. Docente(s)
 - b. Resumo
 - i. Descrição sintética do projeto, entre 300 e 500 palavras
 - c. Contexto
 - i. Problema abordado
 - ii. Motivação
 - iii. Objetivos
 - d. Processo
 - i. Metodologia utilizada
 - ii. Ferramentas utilizadas
 - iii. Tecnologias utilizadas
 - iv. Estrutura da equipa
 - e. Resultados
 - i. Descrição da solução desenvolvida
 - ii. Funcionalidades principais
 - iii. Contributos relevantes
 - f. Reflexão

- i. Lições aprendidas
 - ii. Limitações
 - iii. Trabalho futuro
- 3. Imagens
 - a. Mínimo obrigatório de 10 imagens de alta qualidade. As imagens devem documentar:
 - i. Processo de trabalho
 - ii. Conceitos
 - iii. Protótipos
 - iv. Interfaces
 - v. Mockups
 - vi. Artefactos produzidos
 - vii. Testes
 - viii. Produto final
 - b. Requisitos mínimos para as imagens:
 - i. Formatos vectoriais, quando possível
 - ii. Formatos raster (PNG ou JPEG)
 - 1. Resolução mínima: 3000 px no maior lado ???
 - 2. Sem marcas de água
 - 3. Sem compressão excessiva
- 4. Ficheiros Técnicos
 - a. Arquitetura do sistema
 - b. Diagramas UML
 - c. Estrutura da base de dados
 - d. Capturas da aplicação
 - e. Repositório Git

CONJUNTO ADICIONAL DE DOCUMENTOS

Documentos não obrigatórios, porém contribuem para um arquivo documental mais completo.

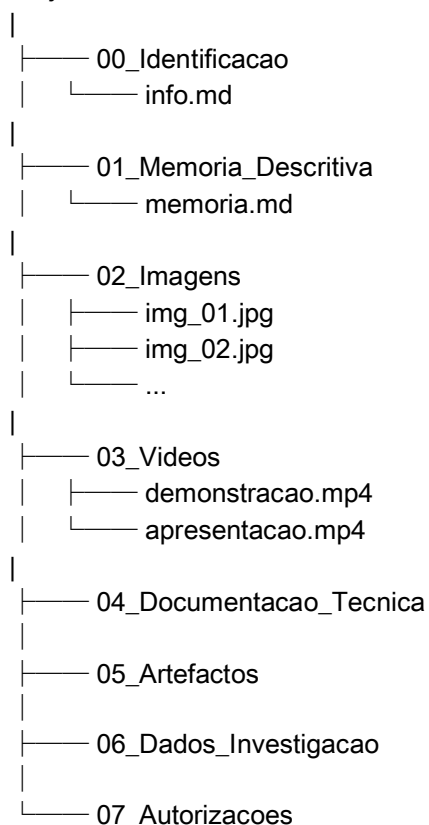
- 1. Fotografias
 - a. Equipa de projeto
 - b. Sessões de trabalho
 - c. Apresentações
 - d. Demonstrações
 - e. Exposições
 - f. Eventos
- 2. Capturas de Ecrã
 - a. Interfaces
 - b. Dashboards
 - c. Ferramentas desenvolvidas
 - d. Sistemas implementados
- 3. Vídeos
 - a. Vídeo curto de demonstração (1 a 3 minutos)
 - b. Vídeo completo da apresentação final
 - c. Formatos recomendados: MP4 (H.264 ou H.265) em Full HD (1080p) ou superior
- 4. Artefactos Digitais
 - a. Repositórios Git
 - b. Ficheiros de design
 - c. Modelos 3D
 - d. Assets gráficos

- e. Bases de dados
- f. Documentação técnica
- 5. Materiais de Investigação
 - a. Revisões de literatura
 - b. Estudos prévios
 - c. Protocolos
 - d. Instrumentos de recolha de dados
 - e. Questionários
 - f. Guiões de entrevista
- 6. Resultados de Avaliação
 - a. Testes de usabilidade
 - b. Dados experimentais
 - c. Métricas de desempenho
 - d. Avaliações de utilizadores

ESTRUTURA DO ARQUIVO

Obrigatoriamente os documentos e pastas listadas acima deverão estar organizados segundo a seguinte estrutura de pastas:

Projeto_Nome/



Falhas ao seguir minuciosamente esta estrutura resultarão na anulação total da pontuação relativa a toda a documentação do trabalho.

Cada projeto deverá incluir um ficheiro de metadados (**info.md**) contendo:

1. Título do projeto
2. Ano letivo
3. Semestre

4. Curso(s) envolvidos
5. Unidade(s) curricular(es)
6. Docente(s)
7. Estudantes participantes
8. Palavras-chave
9. Tecnologias utilizadas
10. Resumo curto (100 a 150 palavras)

Este ficheiro facilitará futuras pesquisas e processos de curadoria.